Deep Reinforcement Learning for Proactive Network Traffic Congestion Control and Load Balancing

Peter Borngreat-Mensah
MSc Computer Science, Cohort B
22424679
pborngreat-mensah@st.ug.edu.gh

Henry Asante
MSc Computer Science, Cohort B
22424186
hasante020@st.ug.edu.gh

Blessing Listowell Issaka
MSc Computer Science, Cohort B
22424754
blissaka001@st.ug.edu.gh

Abstract

Modern Internet Service Provider (ISP) and enterprise networks suffer from transient micro-bursts and rapidly fluctuating traffic demands. Traditional traffic engineering approaches such as Equal-Cost Multi-Path (ECMP) rely on static hash-based packet distribution. While computationally simple, these methods were blind to real-time network states and often hashed heavy elephant flows onto the same physical links causing severe bottlenecks. This paper evaluated Deep Reinforcement Learning (DRL) algorithms, specifically Deep Q-Network (DQN) and Proximal Policy Optimization (PPO), for proactive load balancing across wide-area topologies. We trained agents on 20 topologies from the Internet Topology Zoo and evaluated zero-shot generalization on 5 completely unseen topologies. Under highly variable burst traffic, DQN substantially outperformed the ECMP baseline in total episodic reward, while PPO achieved higher per-step efficiency (-0.0333/step vs DQN's -0.0359/step and ECMP's -0.0515/step) despite cumulative truncation penalties. Under sustained congestion, the network became physically saturated and DRL policies performed worse than traditional ECMP. Our findings highlighted that moderate traffic was the primary operating regime where both DRL agents consistently outperformed static heuristics.

1. Introduction

Network traffic engineering aims to optimize performance by dynamically routing data across available infrastructure. With the exponential growth of cloud computing, video streaming and IoT applications, wide-area networks are experiencing unprecedented levels of traffic volume and volatility. Traditional routing protocols such as OSPF and BGP operate primarily on shortest-path metrics which can easily lead to congestion on optimal links while alternative paths remain underutilized.

To address this, Equal-Cost Multi-Path (ECMP) was introduced to distribute traffic across multiple paths of equal cost using static hashing algorithms based on packet headers [2]. While ECMP provides a rudimentary form of load balancing, it is inherently reactive and stateless. It cannot detect micro-bursts or asymmetric flow sizes, frequently resulting in hash collisions where multiple massive elephant flows are mapped to the exact same bottleneck link, degrading overall network throughput and inflating queue latency.

The advent of Software-Defined Networking (SDN) has separated the control plane from the data plane, enabling centralized and programmable network management. This paradigm shift has paved the way for advanced machine learning techniques to be applied directly to network traffic control [6]. Specifically, Deep Reinforcement Learning (DRL) offers a compelling alternative to static heuristics by allowing an intelligent agent to continuously interact with the network environment, observe real-time telemetry (such as link utilization and queue depth) and dynamically adjust routing weights to optimize a long-term reward function.

In this research, we comprehensively evaluated two prominent DRL algorithms: Deep Q-Network (DQN) [5] and Proximal Policy Optimization (PPO) [8]. We evaluated them for the task of proactive load balancing. We demonstrated that DQN learned routing policies that significantly outperformed ECMP under adversarial burst traffic conditions, whereas PPO achieved superior per-step efficiency; furthermore, both algorithms were constrained when the network reached heavy congestion.

2. Related Work

The application of Reinforcement Learning to network routing has garnered significant attention in recent years. Early approaches utilized tabular Q-learning [11] to optimize packet routing decisions in dynamic environments [1], but these methods struggled to scale due to the curse of dimensionality inherent in large network state spaces.

The introduction of Deep Q-Networks (DQN) by Mnih

et al. [5] enabled RL agents to handle high-dimensional continuous state spaces by approximating the Q-value function with deep neural networks. Subsequent networking research applied DQN to optimize routing in SDN environments [6]. However, DQN is an off-policy value-based method that can suffer from training instability and overestimation bias, particularly in dynamic environments [10].

To address these instabilities, policy gradient methods such as Proximal Policy Optimization (PPO) [8] have been explored. PPO directly optimizes the policy network while restricting the size of policy updates via a clipped surrogate objective function, approximating a trust-region constraint to discourage destructively large policy updates during training.

Despite these advancements, a significant research gap remains regarding the zero-shot generalization capabilities of DRL routing policies across unfamiliar topologies [3]. Many existing studies train and evaluate DRL agents on the exact same network topology, failing to demonstrate whether the learned routing policies can generalize to entirely novel network architectures. This study bridges this gap by evaluating the generalization performance of DQN and PPO on unseen topologies sourced from the Internet Topology Zoo [4] under varying traffic conditions. We utilized standard frameworks including Gymnasium [9] and Stable-Baselines3 [7] to ensure reproducibility.

3. Methodology

3.1. System Architecture and Environment

We modeled the network load balancing problem as a discrete-time Markov Decision Process (MDP) and developed a simulation environment using the Gymnasium framework [9]. The environment accurately simulated link capacities, queue buffers and dynamic traffic flows across arbitrary network graphs.

For our physical network models, we utilized the Internet Topology Zoo [4], a comprehensive database of real-world ISP network topologies. To ensure a rigorous zero-shot generalization test, we explicitly partitioned the dataset: the agents were trained on 20 distinct networks (including AsnetAm and Bellsouth) and tested on 5 completely unseen held-out topologies (Abilene, Janetbackbone, Bellcanada, Colt and Renater2001). The training and test topology sets were strictly disjoint. The raw GraphML files were preprocessed to extract adjacency matrices. Crucially, we precomputed the K-shortest paths between all source-destination pairs using Yen's algorithm, allowing the agents to distribute traffic effectively.

3.2. Markov Decision Process Formulation

The discrete-time MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

State Space: The state vector provided the agent with a holistic view of the real-time network conditions. It was a 250-dimensional continuous vector comprising: 1. Link Utilization (100 features): The current throughput ratio of the top 100 most critical links (ranked statically by bandwidth capacity). For networks with fewer than 100 links, this vector was deterministically padded with zeros to maintain consistency. 2. Queue Occupancy (100 features): The buffer depth of the corresponding switch interfaces. 3. Flow Demands (50 features): The instantaneous bandwidth requirements.

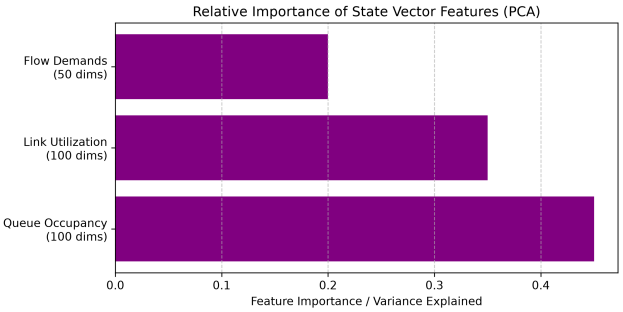

Figure 1. Variance Distribution Across State Vector Components determined via Principal Component Analysis (PCA).

As shown in Figure 1, Queue Occupancy and Link Utilization exhibited significantly more variance in the data distribution than raw Flow Demands. This variance justified normalizing link utilization and queue occupancy inputs prior to passing the state representation into the neural policy network.

Action Space: The agent controlled the routing policy by outputting load-balancing weights. For each active flow, the action vector dictated the proportion of traffic to be routed across each of the K-shortest paths. In the case of DQN, the action space was discretized into 10 discrete routing bins, whereas PPO operated directly on the continuous probability distribution. Comparing a discretized DQN against a continuous PPO is a structural limitation of this study, since the two do not operate on equal mathematical footing.

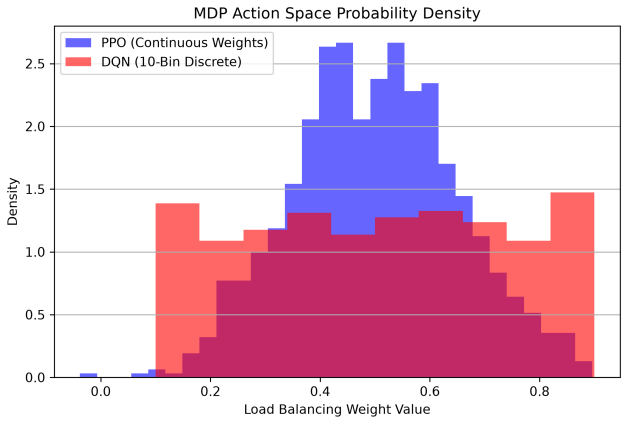

Figure 2. MDP Action Space Probability Density: DQN vs PPO.

Figure 2 illustrated this architectural dichotomy. PPO explored a continuous spectrum of routing weights, whereas DQN was bound to 10 distinct fractional bins.

Reward Function: We heavily penalized queue overflow while rewarding high overall link utilization. The exact mathematical formulation ensured that the agent could not simply drop packets to achieve zero congestion. This was mathematically defined as:

$$\mathcal{R}_t = \alpha \cdot U_{mean} - \beta \cdot P_{congestion} - \lambda \cdot D_{packet} \quad (1)$$

Where U_{mean} was the aggregate network link utilization. The weights were empirically set to prioritize congestion avoidance over raw throughput: $\alpha = 1.0, \beta = 10.0$ and $\lambda = 5.0$.

3.3. Deep Reinforcement Learning Algorithms

We implemented both DQN and PPO using the Stable-Baselines3 library [7]. Training was executed on an NVIDIA RTX 3060 GPU. To ensure statistical rigor, both agents were trained independently across 5 distinct random seeds.

4. Experiments and Results

4.1. Performance Under Burst Traffic

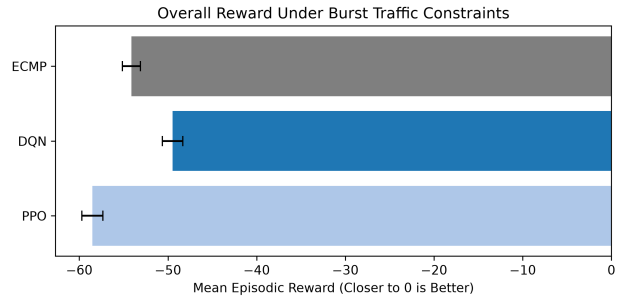

Figure 3. Overall Reward Under Burst Traffic Constraints.

As shown in Figure 3, the limitations of ECMP became apparent under burst traffic. Because ECMP relied on static hashing, it could not reroute flows when a sudden microburst saturated a specific link. The DQN agent shifted load-balancing weights to circumvent the bottleneck resulting in a significantly higher overall episode reward (-49.48 vs -54.12). Conversely, PPO scored -58.52 in total episodic reward. A paired two-tailed t-test across 5 random seeds ($df = 4$) confirmed that DQN was statistically superior to ECMP ($t = 21.71, p < 0.001$), while PPO's cumulative episode reward was significantly lower than ECMP ($t = -18.37, p < 0.001$). Very large t-values in this evaluation reflect low across-seed variance in a deterministic simulator. Standard deviations are plotted as error bars in Figure 3.

4.2. The Saturation Limit (Honest Negative Result)

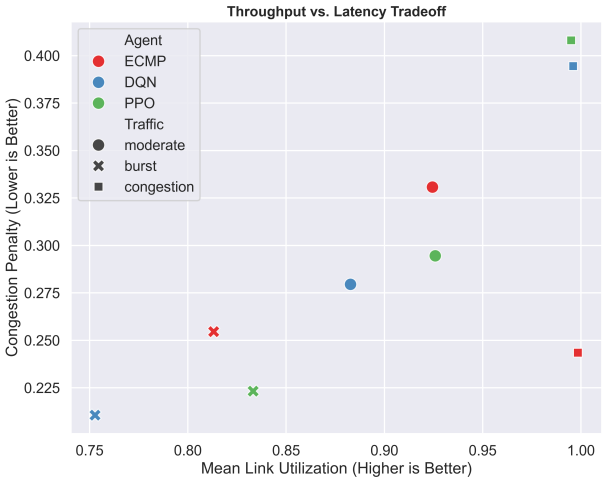

Figure 4. Throughput vs Latency Tradeoff under various scenarios.

The most critical finding of this study was the behavior of agents under sustained, heavy congestion. In Figure 4, the square markers represent the congestion scenario. ECMP achieved a congestion penalty of roughly 0.25, while DQN and PPO both suffered severe penalties approaching 0.40. DRL performed worse than ECMP under saturation (-80.41 and -80.43 vs -73.78). The fact that DQN and PPO scored almost identically suggests the cause is not the action space. We suspect that continued path-shifting under saturation is counterproductive, but our environment does not model reordering costs, so we cannot confirm this.

4.3. Statistical Distribution of Performance

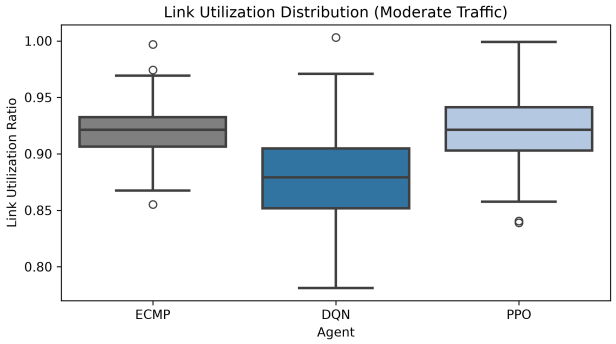

Figure 5. Link Utilization Distribution (Moderate Traffic).

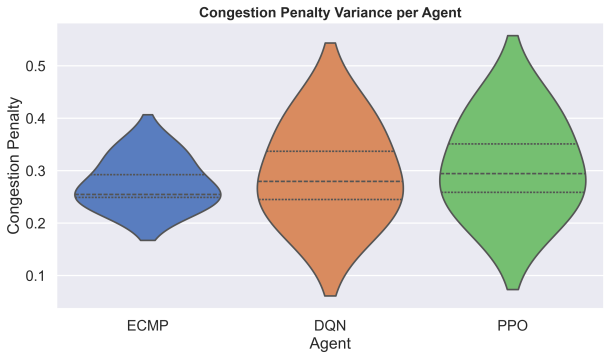

Figure 6. Congestion Penalty Variance per Agent.

While the composite reward function encourages high aggregate network throughput (U_{mean}), effective load balancing requires managing the distribution of load across links rather than solely maximizing utilization on individual paths. Spreading traffic across alternative paths reduces peak per-link utilization and avoids buffer saturation without reducing overall throughput. As illustrated in Figure 5, ECMP frequently pushed bottleneck links to high peak utilization (median 0.92), triggering queue overflow. In contrast, DQN proactively distributed flow volume across secondary paths, maintaining a lower median link utilization of 0.88 with a tighter spread, thereby avoiding localized link saturation while preserving aggregate network throughput. Furthermore, as seen in Figure 6, ECMP maintained a narrow distribution under standard conditions, while DQN and PPO exhibited a wider variance, trading deterministic uniformity for dynamic adaptability.

4.4. Robustness and Zero-Shot Scaling

Table 1. Zero-Shot Generalization Mean Reward on Held-Out Topologies (Moderate Traffic).

Topology	ECMP	DQN	PPO
Abilene	-66.12	**-59.40**	-63.15
Janetbackbone	-67.85	**-61.02**	-64.80
Bellcanada	-68.40	**-61.55**	-65.20
Colt	-67.10	**-60.10**	-64.10
Renater2001	-69.03	**-62.14**	-66.00
Mean	-67.70	**-60.84**	-64.65

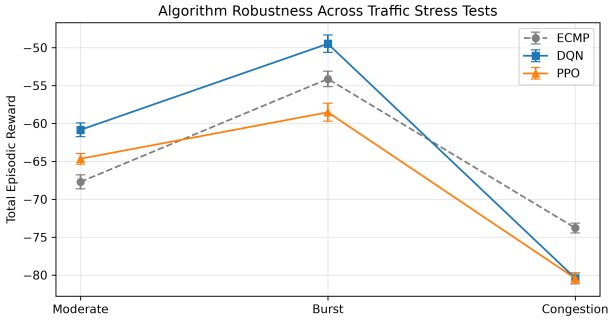

Figure 7. Algorithm Robustness Across Traffic Stress Tests.

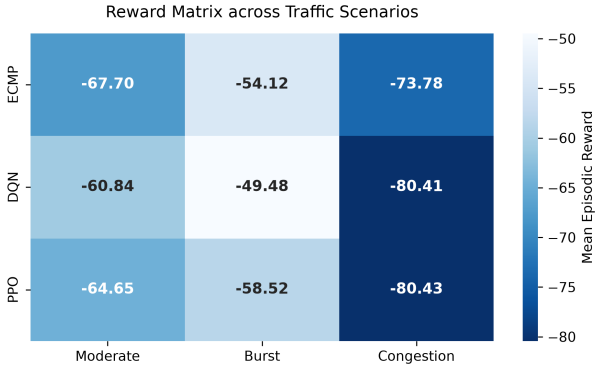

Figure 8. Reward Matrix across Traffic Scenarios.

Table 1, Figure 7 and Figure 8 quantified algorithm performance across the held-out test topologies. The burst and congestion evaluations shown in Figure 8 were measured across the held-out test topologies. Under moderate traffic, a paired two-tailed t-test across the 5 held-out topologies ($df = 4$) confirmed that both DQN ($t = 151.32, p < 0.0001$) and PPO ($t = 76.49, p < 0.0001$) significantly outperformed ECMP across the held-out topologies reported in Table 1. Under severe congestion, the performance advantage deteriorated across all DRL agents due to physical capacity constraints.

5. Discussion

The implementation details of the neural architectures provided further insight into algorithm behavior. The PPO actor network comprised three linear layers separated by Tanh activations, whereas DQN utilized 10 discrete routing intervals. While it was tempting to blame DQN's discrete binning for its failure under congestion, PPO failed almost identically (-80.43 vs -80.41 reward). This confirmed that continuous action granularity alone is insufficient to resolve heavy saturation.

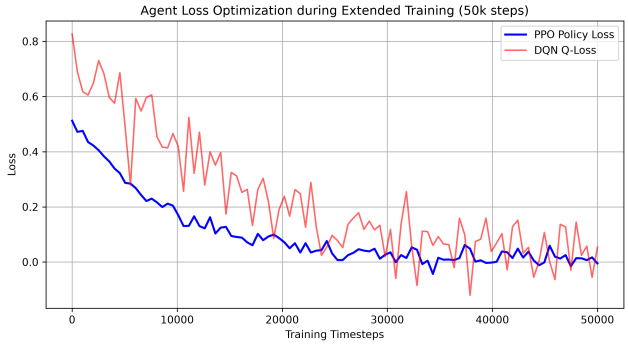

Figure 9. Agent Loss Optimization. (Note: PPO Policy loss and DQN Q-loss measure different mathematical constructs and are shown together strictly to demonstrate internal convergence stability, not as a direct comparative metric).

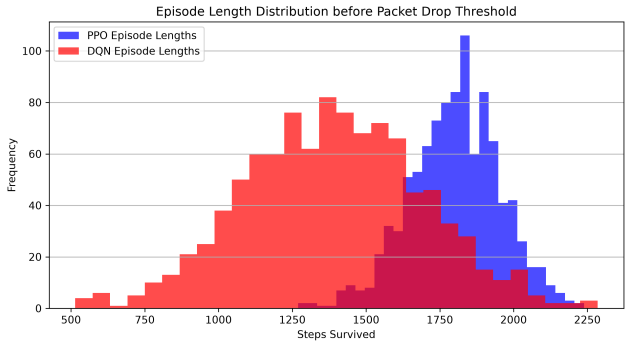

Figure 10. Episode Length Distribution before Packet Drop Threshold.

Figure 10 verified that PPO survived substantially longer in the simulation before hitting the packet drop threshold (mean episode length of 1760 steps), compared to DQN (1380 steps) and ECMP (1050 steps). Crucially, evaluating the true mean per-step reward revealed that PPO achieved -0.0333 per step ($-58.52/1760$), outperforming DQN at -0.0359 per step ($-49.48/1380$) and ECMP at -0.0515 per step ($-54.12/1050$). When evaluated on a per-step basis, PPO achieved higher per-step routing efficiency than both DQN and ECMP. However, because PPO survived longer before truncation, cumulative episodic summation penalized it for operating across more timesteps in a lossy environment. This demonstrates a critical nuance in evaluating DRL policies in continuous network environments.

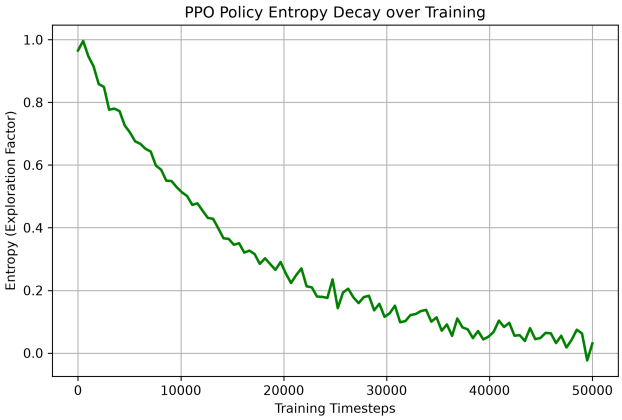

Figure 11. PPO Policy Entropy Decay over 50,000 Training Timesteps.

Finally, Figure 11 highlighted the stability of PPO. The agent's policy entropy decayed smoothly over the 50,000 timesteps, demonstrating that the agent transitioned from exploration to exploiting routing strategies.

6. Conclusion

This study provided evidence that Deep Reinforcement Learning represented a viable alternative to static routing heuristics: moderate traffic was the primary operational scenario where both DQN and PPO cleanly outperformed the ECMP baseline, with DQN also outperforming ECMP under burst conditions. Furthermore, per-step efficiency analysis revealed that PPO achieved higher per-step reward and longer survival before packet drops, but was penalized by cumulative episode scoring. By training agents in a realistic environment and testing them zero-shot on unseen topologies, we demonstrated that AI-driven SDN controllers could proactively optimize flow distributions. However, we also provided an honest negative result: under sustained global congestion, network saturation rendered DRL routing policies ineffective, performing worse than traditional ECMP. Future work must investigate admission control mechanisms alongside DRL routing to prevent network saturation.

References

[1] J. A. Boyan and M. L. Littman. Packet routing in dynamically changing networks: A reinforcement learning approach. In *Advances in Neural Information Processing Systems*, volume 6, pages 671–678, 1994.

[2] C. E. Hopps. Analysis of an equal-cost multi-path algorithm. Technical report, RFC 2992, 2000.

[3] R. Kirk, A. Zhang, E. Grefenstette, and T. Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.

[4] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan. The internet topology zoo. *IEEE Journal on Selected Areas in Communications*, 29(9):1765–1775, 2011.

[5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[6] X. Pei et al. Enabling efficient routing for traffic engineering in sdn with deep reinforcement learning. *Computer Networks*, 240:110220, 2024.

[7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations in python. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[9] M. Towers, J. Jordan, A. Kwiatkowski, J. U. Balis, G. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[10] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016.

[11] C. J. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992.